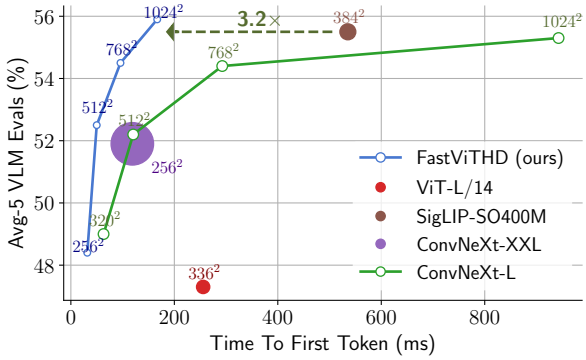
This CVPR paper is the Open Access version, provided by the Computer Vision Foundation.
Except for this watermark, it is identical to the accepted version;
the final published version of the proceedings is available on IEEE Xplore.

FastVLM: Efficient Vision Encoding for Vision Language Models

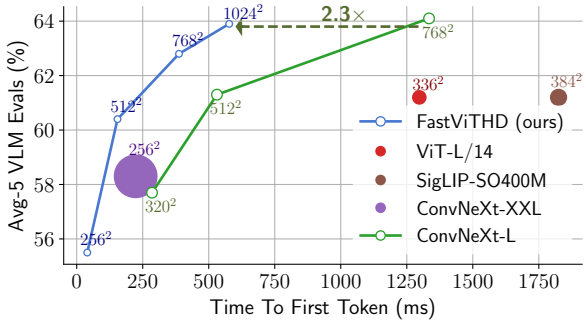
Pavan Kumar Anasosalu Vasu Fartash Faghri Chun-Liang Li Cem Koc Nate True
Albert Antony Gokul Santhanam James Gabriel Peter Grasch Oncel Tuzel
Hadi Pouransari
Apple
{panasosaluvasu,fartash,chunliang_li,cem_koc,otuzel,mpouransari}@apple.com

Abstract

Scaling the input image resolution is essential for enhancing the performance of Vision Language Models (VLMs), particularly in text-rich image understanding tasks. However, popular visual encoders such as ViTs become inefficient at high resolutions due to the large number of tokens and high encoding latency. At different operational resolutions, the vision encoder of a VLM can be optimized along two axes: reducing encoding latency and minimizing the number of visual tokens passed to the LLM, thereby lowering overall latency. Based on a comprehensive efficiency analysis of the interplay between image resolution, vision latency, token count, and LLM size, we introduce FastVLM—a model that achieves an optimized trade-off between resolution, latency, and accuracy. FastVLM incorporates FastViTHD, a novel hybrid vision encoder designed to output fewer tokens and significantly reduce encoding time for high-resolution images. Unlike previous methods, FastVLM achieves the optimal balance between visual token count and image resolution solely by scaling the input image, eliminating the need for additional token pruning and simplifying the model design. In the LLaVA-1.5 setup, FastVLM achieves 3.2× improvement in time-to-first-token (TTFT) while maintaining similar performance on VLM benchmarks compared to prior works. Compared to LLaVa-OneVision at the highest resolution (1152×1152), FastVLM achieves comparable performance on key benchmarks like SeedBench and MMMU, using the same 0.5B LLM, but with 85× faster TTFT and a vision encoder that is 3.4× smaller. Code and models are available at https://github.com/apple/ml-fastvlm.



(a) Qwen2-0.5B



(b) Vicuna-7B

Figure 1. **FastVLM is more than 3× faster than prior work.** Comparison of commonly used vision encoders for VLMs with (a) Qwen2 [80] 0.5B LLM and (b) Vicuna 7B [91] LLM. All the vision encoders are CLIP [64] pretrained. For a fair comparison all models are trained using LLaVA-1.5 [49] setup with the vision encoders made trainable for resolution adaptation, see Sec. 4 for more details. Marker size for each model corresponds to number of parameters of the vision encoder. The x-axis is the sum of vision encoder latency and LLM prefilling time. All models are benchmarked on an M1 Macbook Pro.

1. Introduction

Vision Language Models (VLMs) enable visual understanding alongside textual inputs. VLMs are often built by passing visual tokens from a pretrained vision backbone to a pretrained Large Language Model (LLM) through a projection layer. Previous works [49, 50] have explored various training and fine-tuning strategies for these three components: the vision backbone, the projection, and the LLM, which is typically a decoder-only model.

Several studies [25, 57, 61] highlight image resolution as a key factor in VLM performance, especially for text- and chart-rich data. However, increasing image resolution presents multiple challenges. First, pretrained vision encoders may not support high-resolution images, as this would make pretraining inefficient. To address this, one approach is to continuously pretrain the vision backbone to adapt it for high resolutions [6]. Alternatively, tiling strategies, such as Sphinx [48], S2 [67], and AnyRes [49], divide images into subregions, with each subregion processed independently by the backbone.

A further challenge is the runtime computational cost associated with high-resolution inference. Both single high-resolution inference and multiple inferences at lower resolution (the tiling strategy) result in significant latency when generating visual tokens. Additionally, high-resolution images naturally produce more tokens, which increases the LLM prefilling time (the LLM forward pass time on all tokens in the context, including visual tokens), thereby further increasing the time-to-first-token (TTFT), which is the sum of the vision encoder latency and the LLM prefilling time.

In this work, we study VLM design and training from a runtime efficiency perspective. We explore the optimization landscape as image resolution increases, aiming to improve accuracy-latency trade-off, where latency includes both the vision encoder inference time and the LLM prefilling time. Using extensive experiments with different LLM sizes and resolutions, we establish the Pareto optimal curve for a specific vision backbone, showing the best accuracy achievable within a given runtime budget (TTFT) based on different choices of resolution and LLM size.

We start by exploring the use of a hybrid convolutional-transformer architecture FastViT [77], pretrained with MobileCLIP [78], as a vision backbone for the VLM setup (Section 3.1). We demonstrate the potential of this hybrid backbone, which generates visual tokens over 4× faster than a ViT model while achieving higher overall VLM accuracy with multi-scale features (Section 3.1.1). However, further architectural optimization is possible when the primary goal is a high-resolution VLM (rather than embedding generation as in MobileCLIP-pretrained FastViT). We introduce a new hybrid vision encoder, FastViTHD, specifically designed for efficient VLM performance on high-resolution images (Section 3.2), and use it as the vision backbone to obtain FastVLM through visual instruction tuning. FastVLM demonstrates a significantly improved accuracy-latency trade-off over VLMs based on ViTs, convolutional encoders, and our previously discussed hybrid FastViT for different input image resolutions and LLM sizes (Figure 1a, Figs. 1b and 4). In particular, FastVLM outperforms several prior works while being smaller, faster, and trained with less data (Table 6). Compared to LLaVA-OneVision [41] operating at the highest possible resolution (1152×1152), FastVLM obtains comparable performance with the same 0.5B LLM, but with 85× faster TTFT and a 3.4× smaller vision encoder.

The following is a summary of our contributions:

- We show that hybrid vision backbones outperform ViTs in VLMs, and introduce additional architectural interventions, such as multi-scale vision features, to further improve VLM performance while maintaining efficiency.
- We design and pretrain a new hybrid architecture, FastViTHD, optimized for efficient VLM performance with high resolution input for FastVLM. In a controlled experimental setup, where only the vision backbone is changed, we show that FastViTHD outperforms its ViT-based and convolution-based counterparts when used in VLMs: achieving 3.2× faster TTFT and 3.6× smaller size than SigLIP-SO400M [88], and 2.3× faster TTFT and 1.7× smaller size than ConvNeXT [25]. We further demonstrate that FastVLM scales effectively as more visual instruction tuning data becomes available.
- We systematically study the VLM accuracy-latency trade-off by considering both the vision backbone latency and the LLM prefilling time on actual hardware benchmarks. Our results demonstrate an improved resolution-latency-accuracy trade-off achieved by FastVLM, measured on-device rather than estimates.

2. Related Works

Large Multimodal Models. With the emergence of large language models [63, 72, 74, 80, 91] and large pretrained vision models, such as CLIP [64], trained on web-scale image-text datasets, several multimodal architectures have been proposed to encode images aligned with a large language model (LLM) to enable the interpretation of visual signals. Earlier works like Frozen [75] and Florence [1, 2] used a cross-attention mechanism where the image embeddings are fused with text embeddings in intermediate layers of the LLM. More recently, auto-regressive architectures have gained popularity where the image embedding is fed alongside text as input to an LLM. Some prominent works that use this architecture are LLaVA [49–51], mPLUG-Owl [82–84], InstructBLIP [19], BLIP-3 [79], SPHINX [48], MiniGPT-4 [92], VILA [46], MM1 [61], Qwen-VL [4], InternVL [14, 15] and Cambrian-1 [73]. Recently, Fuyu [5] and EVE [21] introduced a simplified architecture that passes raw images directly to the LLM decoder. Chameleon [71] introduced early fusion mixed-modal models where images are tokenized using a pretrained codebook. While skipping the image encoder is an intriguing approach, the performance of this new class of models lags behind architectures that use a pretrained image encoder.

Efficient Image Encoding. CLIP [64] pretrained vision transformers [22] are widely used for encoding images in vision-language models, with popular choices including

SigLIP [88], EVA-CLIP [70], InternViT [14] and DFN-CLIP [23]. To enhance performance, recent works [32, 68, 73] employ ensembles of vision encoders trained with different objectives. These works are orthogonal to our work as they can benefit from using an efficient vision encoder among the ensemble of vision encoders. Since ViT-based architectures are a popular choice for VLMs, inefficiencies arise from the number of visual tokens, prompting methods like LLaVA-PruMerge [65] and Matryoshka-based token sampling [7, 28] to dynamically prune tokens. Other approaches [9, 17–19] reduce tokens using perceiver-style resamplers or pooling techniques. Rather than using an isotropic architecture like ViT and then designing custom resamplers and projectors, hierarchical architectures can be a simpler design choice. Hierarchical backbones like ConvNeXT [53] and FastViT [77] produce fewer tokens as they downsample the input tensor at every stage of compute. Recently, ConvLLaVA [25] was introduced that uses a pure-convolutional vision encoder to encode images for a VLM. In our work, we introduce an improved convolution-transformer hybrid architecture for VLMs and discuss the pareto-optimal operating points when this architecture is scaled to higher input resolutions.

3. Architecture

In this section, we first explore the adoption of the FastViT hybrid vision encoder for vision-language modeling. We then introduce architectural interventions to improve performance on VLM tasks. We present FastViTHD, a new hybrid vision encoder designed for efficient high-resolution VLM. We provide comprehensive ablations to demonstrate the optimality of FastViTHD over FastViT and prior works for different LLMs and input resolutions. Figure 2 illustrates the overall architecture of FastVLM and FastViTHD. The training setup for all results in this section follows the same configuration as LLaVA-1.5 [49] with Vicuna-7B [91] as the LLM decoder, unless mentioned otherwise. See Sec. 4 for more details.

3.1. FastViT as VLM Image Encoder

VLMs such as LLaVA have three main components: an image encoder, a vision-language projector, and a large language model (LLM). Both the performance and runtime efficiency of a VLM highly depend on its vision backbone. Encoding images at high resolution is essential for achieving strong performance across various VLM benchmarks, especially for text-rich tasks. Therefore, a vision encoder with scalable resolution is particularly beneficial for VLMs. We identify hybrid vision encoders (convolutional layers followed by transformer blocks) as an ideal candidate for VLMs, as their convolutional component enables native resolution scaling, and their transformer blocks further refine high-quality visual tokens for consumption by the LLM.

Image Encoder	Input Res.	#Visual Tokens	Latency Enc.(ms)↓	GQA	TextVQA	POPE	DocVQA	Seed Bench^I	Avg-5
ViT-L/14	336	576	127.4	62.0	58.2	85.9	28.1	66.1	60.1
ViT-L/14†	336	576	127.4	63.5	59.2	86.3	28.7	68.6	61.2
FastViT	256	64	3.0	60.2	51.6	82.9	15.8	61.5	54.4
FastViT	768	576	34.5	62.7	62.3	86.5	34.4	67.1	**62.6**

Table 1. **FastViT has higher accuracy than ViT-L/14 at near 4× lower latency.** To scale resolution up to 768, FastViT is made trainable during Stage-2 training of LLaVA-1.5 setup. †To have a fair comparison, we also report the performance of ViT-L/14 fine-tuned during Stage-2 training of LLaVA-1.5. All latencies are reported in milliseconds. See Sec. 4 for details.

Image Encoder	Multi Scale	Pool Type	GQA	TextVQA	POPE	DocVQA	Seed Bench^I	Avg-5
FastViT		-	62.7	62.3	86.5	34.4	67.1	62.6
FastViT	✓	AvgPool	63.0	62.2	86.2	35.1	66.9	62.7
FastViT	✓	DWConv	63.0	62.5	86.8	34.7	67.4	62.9

Table 2. **Pushing FastViT VLM performance using multi-scale features and pooling strategies.** These modifications slightly improve FastViT. Training setup is LLaVA-1.5 with Vicuna 7B.

We use a CLIP-pretrained hybrid vision encoder, specifically the MCi2 image encoder from MobileCLIP [78], which has 35.7M parameters and is based on the FastViT architecture. For simplicity, we refer to this encoder as "FastViT" throughout the rest of the paper. As shown in Tab. 1, using FastViT at its CLIP-pretrained resolution (256×256) alone does not yield a strong VLM. The main advantage of a hybrid encoder like FastViT lies in its favorable image resolution scaling characteristics, meaning it generates 5.2× fewer tokens than the ViT architecture with a patch size of 14. The token reduction gives significant advantage to VLM, as it reduces the prefilling time and time-to-first-token of the transformer decoders. When the input resolution of FastViT is scaled to 768×768, it produces the same number of visual tokens as ViT-L/14 with an input resolution of 336×336 but achieves better performance on VLM benchmarks. This performance gap is even more pronounced on text-rich benchmarks like TextVQA and DocVQA, despite both architectures producing the same number of visual tokens. Moreover, even with the same token count at higher resolution, it encodes images much faster due to efficient convolution layers.

3.1.1. Multi-Scale Features

Typical convolutional and hybrid architectures split up the computations into 4 distinct stages with a downsampling operation between them. While the VLM relies on features from the penultimate layer, features in earlier stages of the network extract information at different granularity. Aggregating information from multiple scales can complement high-level features from the penultimate layer. The architecture for multiple scale feature extraction is shown in Fig. 2. We ablate between 2 designs to pool features

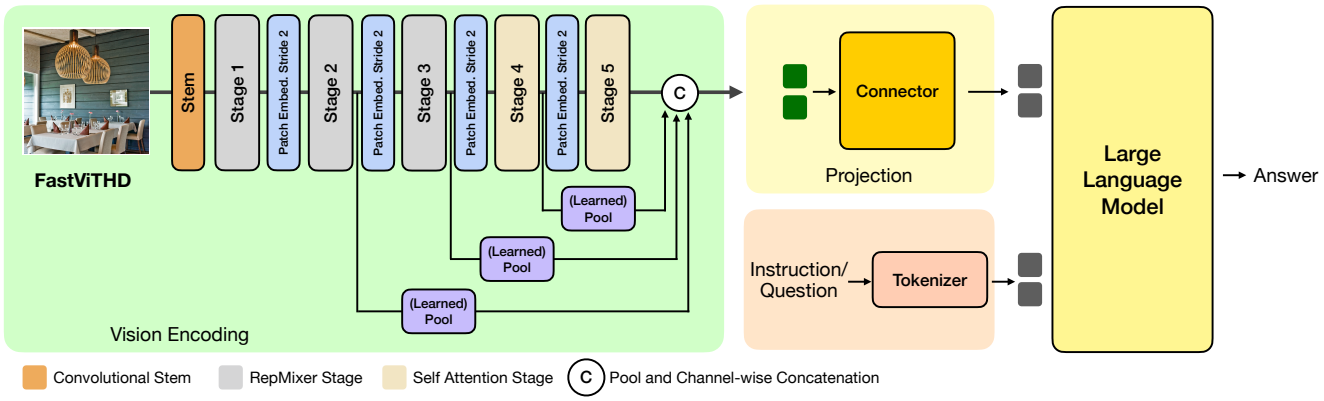


Figure 2. **Overview of the FastVLM architecture.** FastVLM consists of our novel vision encoder, FastViTHD, trained using the same setup as LLaVA. The FastViTHD architecture is designed for low latency at high resolution, by utilizing additional self-attention layers, and downsampling to generate 4× fewer tokens than FastViT, and 16× fewer tokens than ViT-L/14 at resolution 336.

from different stages, i.e. AvgPooling and 2D Depthwise convolutions. From Tab. 2, we find that using depthwise convolutions results in better performance.

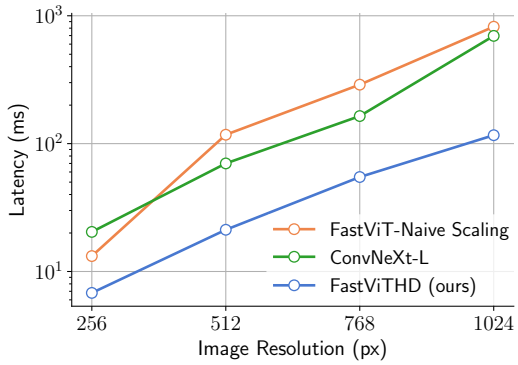


Figure 3. **Novel scaling strategy of FastViTHD lowers latency at various image resolutions.** FastViT-Naive, a naive scaling of the FastViT architecture, and our proposed FastViTHD have the same number of parameters. ConvNeXt-L is provided for reference. All models are benchmarked on M1 Macbook Pro and trained with LLaVA-1.5 setup and Vicuna 7B. Note that the y-axis is in log scale.

3.2. FastViTHD: High Resolution Encoder for VLM

While FastViT with the introduced model interventions performs well as an image encoder that is 8.7× smaller than ViT-L/14, previous studies [14, 43] have demonstrated that increasing the scale of the image encoder improves its generalization capabilities. Hybrid architectures [16, 86] typically scale the number of self-attention layers and width in a 4-stage design, but this has drawbacks. From Fig. 3, simply scaling-up the number of self-attention layers in stages 3 and 4 of FastViT, as done in prior works, is suboptimal and slower than ConvNeXT-L. To mitigate this, we introduce an extra stage with a downsampling layer, ensuring self-attention operates on tensors downsampled by a factor

Image Encoder	Encoder Size(M)↓	Input Res.	Latency Enc.(ms)↓	Zero-Shot ImageNet	Avg Perf. Retrieval	Avg Perf. on 38 tasks
ViT-L/14 [24]	304	224	47.2	79.2	60.8	66.3
ViTamin-L [11]	333	224	38.1	80.8	60.3	66.7
ConvNeXt-L	200	320	34.4	76.8	64.8	63.9
FastViTHD	125	224	6.8	78.3	67.7	66.3

Table 3. **FastViTHD achieves competitive results on CLIP benchmarks at significantly lower latency.** We follow the same setup described in [11] to report average retrieval performance and setup described in [24] to report average performance on 38 tasks. All models are benchmarked on M1 Macbook Pro.

of 32, rather than 16 as in recent models like ViTamin, see Fig. 2. More details on the naive scaling approach can be found in Sec. B. Our design reduces image encoding latency and generates 4× fewer tokens for the compute-intensive LLM decoder, thereby decreasing the time-to-first-token (TTFT). The architecture schematic is shown in Fig. 2, and we call this model **FastViTHD**.

The model architecture consists of 5 stages, as shown in Fig. 2, with the first three stages utilizing RepMixer [77] blocks and the last two stages employing multi-headed self-attention [22] blocks. The model depth at each stage is `[2, 12, 24, 4, 2]`, and the embedding dimensions for each stage are `[96, 192, 384, 768, 1536]`. The MLP expansion ratio for the ConvFFN layers is set to 4.0. The model has 125.1M parameters, which is 3.5× larger than the largest FastViT variant from MobileCLIP, but is still smaller than popular ViT alternatives.

We follow the CLIP pretraining setup of [78] using the DataCompDR-1B dataset to pretrain FastViTHD before employing it for FastVLM training. Table 3 shows that FastViTHD, despite being 2.4× smaller and 6.9× faster than ViT-L/14, achieves comparable average performance across 38 multi-modal zero-shot tasks [24]. In comparison to ViTamin [11], a hybrid transformer architecture built for VLMs, FastViTHD delivers superior average retrieval performance while being 2.7× smaller and 5.6× faster. In

Image Encoder	Input Res.	Latency Enc.(ms)↓	#Visual Tokens	GQA	TextVQA	POPE	DocVQA	Seed Bench^I	Avg-5
FastViTHD	256	10.1	16	60.6	53.1	82.3	17.4	63.7	55.5
C.N-L	320	34.4	100	61.9	55.5	85.3	21.3	64.6	57.7
C.N-XXL	256	89.9	64	62.7	56.3	85.3	21.6	65.6	58.3
FastViTHD	512	33.5	64	63.0	59.3	86.4	25.7	67.1	**60.4**
FastViTHD	768	122.6	144	62.4	62.9	87.7	32.9	68.2	62.8
C.N-L	512	71.9	256	61.8	61.0	86.3	30.8	66.8	61.3
C.N-XXL	512	397.1	256	62.3	65.1	87.7	36.2	68.4	**63.9**
FastViTHD	1024	235.6	256	63.1	64.4	88.1	35.6	68.5	**63.9**

Table 4. **FastViTHD achieves higher accuracy than ConvNeXT while having lower latency at a higher resolution.** The models are grouped based on the total number of visual tokens produced for the LLM to process. "C.N" stands for ConvNeXT. Training setup is LLaVA-1.5 with Vicuna 7B.

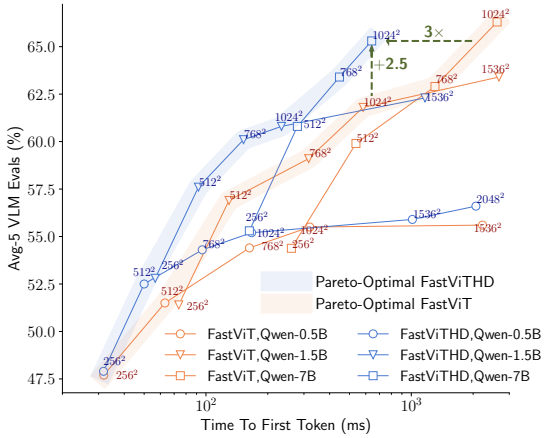


Figure 4. **FastViTHD improves the Pareto-Optimal curve for accuracy versus time to first token compared with FastViT.** Comparison of FastViT and FastViTHD backbones paired with Qwen2 [80] family (chat variant) LLMs of varying sizes and different image resolutions (annotated for each point). The Pareto-optimal curve is highlighted for the two vision backbones. Training setup is LLaVA-1.5. Note that the x-axis is in log scale.

Tab. 4, we compare FastViTHD with other CLIP-pretrained hierarchical backbones, i.e. ConvNeXT-L and ConvNeXT-XXL, for VLM tasks after LLaVA-1.5 training. FastViTHD performs as well as ConvNeXT-XXL while being 6.8× smaller and 3.3× faster.

3.2.1. Vision Encoder - Language Decoder Interplay

The accuracy-latency trade-off in a VLM is influenced by several factors. On one hand, the overall performance of the VLM depends on (1) the input image resolution, (2) the quantity and quality of visual tokens, and (3) the capability of the LLM. On the other hand, the total latency (time to first token generation) of a VLM is determined by (1) the latency of the vision encoder and (2) the prefilling time of the LLM. The latter is affected by both the number of tokens produced by the vision encoder and the size of the LLM.

Due to the complex optimization landscape of VLMs, claims regarding the optimality of a vision encoder must be

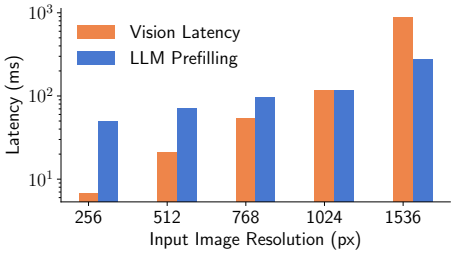


Figure 5. **Vision latency dominates at high resolution.** Breakdown of FastVLM's time to first token for varying image resolutions. Vision encoder is FastViTHD and LLM is Qwen2-1.5B.

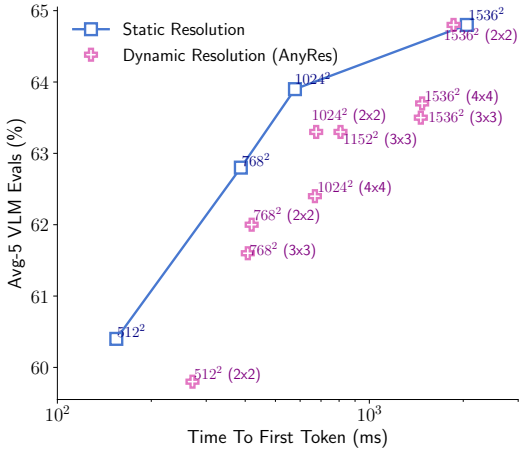


Figure 6. **Dynamic input resolution (AnyRes) is only optimal at the highest resolution when using fewer tiles (2×2).** The vision encoder is FastViTHD. The tile grid size is specified in parenthesis. Training setup is LLaVA-1.5 with Vicuna 7B. Note that the x-axis is in log scale.

verified across various pairs of (Resolution, LLM). Here, we empirically demonstrate the optimality of FastViTHD over FastViT. For each vision encoder, we consider three LLMs, Qwen2 [80]-0.5B/1.5B/7B, along with a range of input image resolutions. For each (Resolution, LLM) pair, we conduct LLaVA-1.5 [49] pretraining and visual instruction tuning, and evaluate the resulting model over a range of tasks. The results are presented in Fig. 4.

First, we observe that for a vision encoder, the Pareto-optimal curve (highlighted in Fig. 4), which represents the maximum achievable performance for a given runtime budget (TTFT), consists of varying sizes of LLMs. Specifically, pairing high resolution with a small LLM is suboptimal as a small LLM cannot effectively utilize that many tokens, and TTFT will be dominated by the latency of the vision encoder (see Fig. 5).

Second, the Pareto-optimal curve for FastViTHD in Fig. 4 is significantly better than that of FastViT. For a given runtime budget, considering all possible (Resolution, LLM) pairs, we achieve significantly better performance (an improvement of over 2.5 points on the Average-5 metric)

with FastViTHD. Similarly, FastViTHD can reach a target VLM performance up to 3× faster. It is important to note that in previous sections, we demonstrated that a FastViT-based VLM already represents a significant improvement over ViT-based VLMs, and yet FastViTHD provides substantial gains over FastViT.

3.2.2. Static vs. Dynamic Input Resolution

There are two ways to scale input resolution: adjusting the model's input resolution directly or tiling the image and setting the encoder's resolution to the tile size. The tiled inference (AnyRes) was introduced in prior works [48, 50] to enable ViT models to process high resolution images. Since FastViTHD is designed to run inference efficiently on high input resolutions, we analyze the optimal operating point for various resolutions using the two strategies. From Fig. 6, we see that setting the model's input resolution directly to the desired resolution offers the best accuracy-latency tradeoff, with dynamic resolution benefiting only at extreme resolutions like 1536×1536, due to memory bandwidth limitations. If dynamic resolution is desired, using a setting with fewer tiles exhibits better accuracy-latency tradeoff. Further discussion on this setup is presented in Sec. C.1.

3.2.3. Comparison with Token Pruning & Downsampling

We further compare the performance of FastViTHD operating at different resolutions to popular token pruning methods in literature. From Tab. 5, we find that VLMs achieve better accuracy to latency trade-off using a hierarchical backbone as opposed to using token pruning methods on isotropic architectures like ViT. By simply training the VLMs at lower input resolution, FastViTHD achieves visual token counts as low as 16, while improving over recent token pruning methods. Interestingly, even the most effective token pruning methods, such as those proposed by [7, 28, 29, 81], perform worse than FastViTHD trained at a lower input resolution of 256×256.

Model	Input Res.	#Visual Tokens	GQA	SQA	Text-VQA	POPE	VQA v2	Seed Bench
ViT-L/14 M^3 [7]	336	9	58.0	-	-	**83.4**	-	55.4
ViT-L/14 MQT [28]	336	16	57.6	67.5	-	80.8	71.1	-
FastViTHD	256	16	**60.6**	**69.2**	53.1	82.3	**74.7**	**58.8**
ViT-L/14 PruMerge [65]	336	40	-	68.5	56.0	76.3	72.0	-
ViT-L/14 PruMerge+ [65]	336	40	-	68.3	57.1	84.0	76.8	-
ViT-L/14 M^3 [7]	336	36	60.3	-	-	85.5	-	58.0
FastV [13]	336	64	46.1	51.1	47.8	48.0	55.0	51.9
SparseVLM [90]	336	64	52.7	62.2	51.8	75.1	68.2	51.1
VisionZip [81]	336	64	55.1	69.0	55.5	77.0	62.9	52.2
VisionZip‡ [81]	336	64	57.0	68.8	56.0	80.9	74.2	53.4
DynamicLLaVA$_I$ [29]	336	115	61.4	**69.1**	57.0	85.0	**78.0**	-
DynamicLLaVA$_{I\|T}$ [29]	336	115	61.3	68.6	56.5	85.9	77.9	-
FastViTHD	512	64	**63.0**	68.9	**59.3**	**86.4**	**78.0**	**61.8**
ViT-L/14 M^3 [7]	336	144	61.3	-	-	87.0	-	59.7
ViT-L/14 MQT [28]	336	144	61.4	67.6	-	83.9	76.4	-
FastV [13]	336	192	52.7	67.3	52.5	64.8	67.1	57.1
SparseVLM [90]	336	192	57.6	**69.1**	56.1	83.6	75.6	55.8
VisionZip [81]	336	192	59.3	68.9	57.3	85.3	76.8	56.4
VisionZip‡ [81]	336	192	60.1	68.2	57.8	84.9	77.4	57.1
FastViTHD	768	144	**62.4**	67.6	**62.9**	**87.7**	**78.9**	**62.5**
ViT-L/14 MQT [28]	336	256	61.6	**67.5**	-	84.4	76.8	-
FastViTHD	1024	256	**63.1**	67.4	64.4	**88.1**	**79.2**	-

Table 5. **FastViTHD more effectively reduces tokens compared with token pruning methods.** The models are grouped based on total number of visual tokens. "-" indicates that performance was not reported in the respective paper. All models presented in this table are trained using LLaVA-1.5 setup with Vicuna 7B. ‡- indicates further finetuning as reported in [81]. $_I$ - indicates vision only sparsification and $_{I|T}$ indicates vision-language sparsification, as reported in [29].

4. Experiments

Training Setup. For all the ablations presented in Sec. 3, we follow the 2-stage setup described in LLaVA-1.5 [49] with Vicuna-7B [91] as the LLM decoder, unless mentioned otherwise. During the first stage, only the projector is trained using LLaVA-558K alignment dataset for one epoch, with a batch size of 256 and a learning rate of 10^{-3}. At this stage, the input image resolution matches the backbone pretraining resolution (e.g., 256 for FastViT and 224 for FastViTHD). In the second stage, we use LLaVA-665K supervised finetuning dataset, training the models for one epoch and tuning all the modules, i.e., vision encoder, projector and the LLM. At this stage, the input image resolution is set to the target resolution.

In Sec. 4, we present results with different LLM decoders, primarily with Qwen2-0.5B/1.5B/7B model family [80] (chat variant) and Vicuna-7B model [91]. We report results in two training setups, the first one is the 2-Stage setup introduced in LLaVA-1.5. For the second training setup, we follow the current trend in literature [39, 61] of training the VLMs in 3 stages, i.e. Stage 1 for training the connector, Stage 1.5 for resolution scaling and Stage 2 for visual instruction tuning. Information on datasets used in these stages can be found in Sec. D. In this setup, the input image resolution is set to the backbone pretraining resolution for Stage 1 and adjusted to the target resolution for the following two stages. In both setups, the vision encoder and LLM are frozen only in stage 1, while all modules are finetuned in the remaining stages.

All FastVLM models reported in the paper are trained on a single node with 8× NVIDIA H100-80GB GPUs. Stage 1 training of VLM is quick, taking roughly 30 minutes to train with a Qwen2-7B decoder. Stage 1.5 and Stage 2 training runs are dependent on input resolution. For an input resolution of 1024×1024, Stage 1.5 takes 77 hours and Stage 2 takes 8 hours. The reported wall clock times correspond to the following datasets used in these stages: 15 million samples in Stage 1.5 and 1.1 million samples in Stage 2.

Evaluation. We evaluate the models on the mainstream benchmarks of GQA [30], ScienceQA [55], TextVQA [69], POPE [44], LLaVA-in-the-wild [50],

Row Ann.	Method	Vision Encoder	LLM	Data (M) (PT+IT)	Input Res.	#Visual Tokens	Vis. Enc. Size(M)↓	TTFT (ms)↓	GQA	SQA	Text VQA	POPE	LLaVA Bench^W	MM-Vet	VQA v2	Doc VQA	MMMU	Seed Bench^I
0.5B Model Comparison																		
R1	nanoLLaVA	ViT-SO400M	Qw.1.5	-	384	729	430	535	54.8	59.0	46.7	84.1	-	-	70.8	-	30.4	-
R2	LLaVAOV [41]*	ViT-SO400M	Qw.2	4.5+3.2	1152	7290	430	14124	-	67.2	-	-	-	29.1	-	70.0	31.4	65.5
R3	**FastVLM (Ours)**	FastViTHD	Qw.2	15+1.1	1024	256	**125**	**166**	61.6	61.4	57.4	**87.4**	56.0	31.8	77.0	61.0	30.9	65.6
R4	**FastVLM (Ours)**	FastViTHD	Qw.2	15+12.5	1024	256	**125**	**166**	63.1	**81.5**	62.9	86.6	66.7	29.8	78.8	70.4	**32.9**	69.2
R5	**FastVLM (Ours)***	FastViTHD	Qw.2	15+12.5	2048	1280	**125**	918	**64.0**	80.7	**65.4**	86.9	65.2	**35.3**	**79.2**	**82.1**	**32.9**	**69.6**
1-2B Model Comparison																		
R6	MobileVLMv2 [18]	ViT-L/14	ML.	1.2+3.6	336	144	304	458	59.3	66.7	52.1	84.3	-	-	-	-	-	-
R7	**FastVLM (Ours)**	FastViTHD	Qw.2	15+1.1	768	144	**125**	**152**	63.9	75.8	64.4	87.2	65.2	35.4	79.4	61.3	34.9	71.7
R8	**FastVLM (Ours)**	FastViTHD	Qw.2	15+12.5	768	144	**125**	**152**	**64.2**	**87.9**	**66.4**	**88.3**	**68.5**	**41.1**	**80.1**	**69.4**	**38.1**	**72.4**
R9	DeepSeekVL [54]	ViT-SO400M	DS.	-	384	576	430	-	-	-	-	87.6	-	34.8	-	-	32.2	66.7
R10	MM1 [61]*	ViT-H	-	3000+1.5	1344	720	632	-	-	62.3	68.2	87.4	67.5	39.4	-	68.4	33.2	65.6
R11	**FastVLM (Ours)**	FastViTHD	Qw.2	15+1.1	1024	256	**125**	**233**	64.2	74.8	66.0	**88.0**	66.9	37.6	79.9	67.7	33.1	71.4
R12	**FastVLM (Ours)**	FastViTHD	Qw.2	15+12.5	1024	256	**125**	**233**	64.3	89.9	69.0	87.8	68.0	43.6	80.7	75.6	39.1	73.1
R13	**FastVLM (Ours)***	FastViTHD	Qw.2	15+12.5	2048	1280	**125**	1263	**64.7**	**90.8**	**71.5**	86.8	**71.3**	**45.2**	**81.0**	**87.6**	**40.1**	**73.4**
7B Model Comparison																		
R14	InstructBLIP [19]	ViT-g/14	Vic.	129+1.2	224	32	1012	302	49.2	60.5	50.1	-	60.9	26.2	-	-	30.6	-
R15	**FastVLM (Ours)**	FastViTHD	Vic.	0.5+0.6	256	16	**125**	**150**	60.6	69.2	53.1	82.3	60.4	27.5	74.7	17.4	36.2	63.7
R16	**FastVLM (Ours)**	FastViTHD	Vic.	15+1.1	256	16	**125**	**150**	**62.1**	**75.7**	**57.2**	**83.9**	**64.0**	**31.5**	**77.3**	**29.8**	**37.6**	**68.8**
R17	MobileVLMv2 [18]	ViT-L/14	Vic.	1.2+3.6	336	144	304	460	62.6	74.8	62.3	85.3	-	-	-	-	-	-
R18	ConvLLaVA [25]	ConvNeXT-L	Vic.	4.9+0.6	768	144	200	496	-	-	59.1	87.3	-	44.8	-	44.8	36.3	68.8
R19	**FastVLM (Ours)**	FastViTHD	Vic.	0.5+0.6	768	144	**125**	**387**	62.4	67.6	62.9	**87.7**	63.8	31.5	78.9	32.9	34.9	68.2
R20	**FastVLM (Ours)**	FastViTHD	Vic.	0.5+1.1	768	144	**125**	**387**	63.2	73.5	67.5	86.3	63.9	33.0	79.1	57.3	36.9	69.9
R21	**FastVLM (Ours)**	FastViTHD	Vic.	15+1.1	768	144	**125**	**387**	65.0	78.7	69.4	87.5	67.0	42.2	81.3	65.5	37.0	73.7
R22	**FastVLM (Ours)**	FastViTHD	Qw.2	15+1.1	768	144	**125**	446	**65.6**	85.9	69.5	87.2	73.0	41.3	**81.3**	66.9	43.6	**75.3**
R23	**FastVLM (Ours)**	FastViTHD	Qw.2	15+12.5	768	144	**125**	446	64.7	**96.0**	**71.9**	87.4	**73.6**	**49.4**	81.1	**75.3**	**44.6**	74.3
R24	Qwen-VL [4]	ViT-G/14	Qw.	1400+50	448	256	1844	-	59.3	67.1	63.8	-	-	-	79.5	65.1	-	-
R25	Qwen-VL-Chat [4]	ViT-G/14	Qw.	1400+50	448	256	1844	-	57.5	68.2	61.5	-	-	-	78.2	62.6	-	-
R26	ConvLLaVA [25]	ConvNeXT-L	Vic.	4.9+0.6	1024	256	200	1157	-	-	62.5	87.7	-	**44.4**	-	48.5	35.1	69.3
R27	**FastVLM (Ours)**	FastViTHD	Vic.	0.5+0.6	1024	256	**125**	**577**	63.1	67.4	64.4	**88.1**	64.8	31.7	79.2	35.6	35.1	68.5
R28	**FastVLM (Ours)**	FastViTHD	Vic.	0.5+1.1	1024	256	**125**	**577**	63.3	74.1	67.4	87.1	66.5	32.4	79.3	62.8	37.3	69.9
R29	**FastVLM (Ours)**	FastViTHD	Vic.	15+1.1	1024	256	**125**	**577**	65.2	80.3	70.6	87.2	71.5	40.1	81.6	72.4	36.7	73.5
R30	**FastVLM (Ours)**	FastViTHD	Qw.2	15+1.1	1024	256	**125**	641	**65.8**	**84.9**	**72.1**	87.8	**75.8**	44.1	**81.7**	**73.3**	**46.2**	**75.1**
R31	LLaVA-1.5 [49]	ViT-L/14	Vic.	0.5+0.6	336	576	304	1297	62.0	70.4	58.2	85.9	59.6	31.1	76.6	28.1	35.3	66.1
R32	MobileVLMv2 [18]	ViT-L/14	Vic.	1.2+3.6	336	576	304	1297	64.6	74.8	66.8	86.1	-	-	-	-	-	-
R33	ShareGPT4V [12]	ViT-L/14	Vic.	1.2+0.7	336	576	304	1297	63.3	68.4	60.4	85.7	72.6	37.6	80.6	-	-	69.7
R34	ViTamin [11]	ViTamin-L	Vic.	0.5+0.6	384	576	333	1308	61.6	67.6	59.8	85.5	66.1	33.6	78.9	-	-	-
R35	ConvLLaVA [25]	ConvNeXT-L	Vic.	4.9+0.6	1536	576	200	2740	-	-	65.8	87.3	-	45.9	-	59.0	35.8	70.2
R36	VILA [46]	ViT-L/14	L-2	50+1	336	576	304	1297	62.3	68.2	64.4	85.5	69.7	34.9	79.9	-	-	62.8
R37	LLaVA-FlexAttn [42]	ViT-L/14	Vic.	0.5+0.6	1008	576	304	-	62.2	-	48.9	85.9	-	29.4	78.7	-	-	-
R38	MM1 [61]*	ViT-H	-	3000+1.5	1344	720	632	-	-	72.6	72.8	86.6	**81.5**	42.1	**82.8**	76.8	37.0	69.9
R39	LLaVA-NeXT†*	ViT-L/14	L-3	-	672	2880	304	20347	65.2	72.8	64.6	-	80.1	-	-	78.2	41.7	72.7
R40	**FastVLM (Ours)**	FastViTHD	Qw.2	15+6.5	1024	256	**125**	**641**	**66.0**	87.4	73.1	87.3	72.4	47.6	82.3	78.7	42.8	**75.9**
R41	**FastVLM (Ours)**	FastViTHD	Qw.2	15+12.5	1024	256	**125**	**641**	65.2	95.7	73.4	86.9	71.1	48.4	81.6	82.7	47.3	74.1
R42	**FastVLM (Ours)***	FastViTHD	Qw.2	15+12.5	2048	1280	**125**	3721	65.5	**96.9**	**76.6**	**87.6**	75.1	**54.7**	82.5	**92.3**	**47.6**	75.8
VLMs with Multiple Vision Encoders and 8B LLM																		
R43	MiniGemini-HD†	ConvNeXT-L ViT-L/14	L-3	1.5+1.5	1536 672	2880	200 304	21832	64.5	75.1	70.2	-	-	-	-	74.6	37.3	73.2
R44	Cambrian-1 [73]	ViT-SO400M ConvNeXt-XXL DINOv2-ViT-L/14 ViT-L/14	L-3	2.5+7	384 1024 518 336	576	430 846 304 304	5085	64.6	80.4	71.7	-	-	-	-	77.8	42.7	74.7

Table 6. **VLM evaluations and comparison with recent methods.** The models are grouped based on total number of visual tokens. "-" indicates that performance was not reported in the respective paper. For the dataset column, "-" indicates that the dataset size for pretraining ("PT") or instruction tuning ("IT") is not explicitly mentioned in the respective paper. For methods that have more than 2 stages of training, we report the total samples used for all the pretraining stages as part of "PT". "TTFT" means time to first token (the sum of the vision encoder latency and the LLM prefilling time), we report latency only for models that are publicly available and in a format favorable to MLX [27] "Vic." refers to Vicuna [91], "Qw.2" refers to Qwen2 [80] and "Qw." refers to Qwen [3]. "L-2" refers to LLaMA-2. "L-3" refers to LLaMA-3. "ML." refers to MobileLLaMA [17, 18]. "DS." refers to DeepSeek LLM [20]. * For input resolution and visual tokens, we report the highest supported resolution by the respective models as some models like LLaVA-OneVision [41] and MM1 [61] use dynamic input resolution. FastVLM models using dynamic resolution employs a simple 2×2 grid, with tile size set to 1024. †- performance numbers reported from [73]. For VLMs that use multiple vision encoders, the size of each encoder is listed independently, for TTFT, the latency from each encoder is summed up.

VQAv2 [26], MMVet [85], MMMU [87], DocVQA [60] and SeedBench [38]. For GQA, ScienceQA, TextVQA, POPE and LLaVA-in-the-wild benchmarks, we use the official evaluation from LLaVA [50]. For the remaining evaluations we use lmms-eval [89] library v0.2.2. We use the default settings for all the evaluations and lmms-eval defaults to 0613 version of GPT for evaluations that rely on GPT as a judge.

For ablations presented in Sec. 3, we report GQA, TextVQA, POPE, DocVQA and SeedBench. GQA and SeedBench are general knowledge benchmarks, DocVQA and TextVQA represent text-rich evaluations and POPE is a hallucination benchmark. Together these benchmarks provide diversity and are quick to evaluate for ablations. Most importantly, they exhibit lower variance to different initializations and under probabilistic decoding setting. We report the variance for all the evals for different initialization in Sec. D.3. The standard deviation across the 5 selected metrics is less than 0.5. We call the average of these 5 benchmarks Avg-5, and use it as a reliable signal for our analysis. The empirical standard deviation estimate for Avg-5 is 0.1.

Benchmarking. We benchmark all the models on a MacBook Pro with the M1 Max chip and 32GB RAM. The image encoder is converted to a Core ML package file using coremltools v7.2 and benchmarked on the neural engine using XCode 15.4 (15F31d). The LLM is benchmarked on the MacBook Pro GPU using MLX [27]. The model is first converted using `mlx_lm.convert` tool, which converts the models on huggingface to the MLX format and casts the tensors to FP16. The prefilling latency is estimated using `mlx_lm.cache_prompt` tool [27]. Time-To-First-Token (TTFT) is estimated by adding the image encoding latency at a specific resolution to the LLM prefilling latency for the associated visual tokens.

4.1. Comparison with state-of-the-art

In Tab. 6, we compare FastVLM with recently published methods. The training setup can vary widely between works. For each, we report the LLM decoder and the sizes of the instruction tuning and pretraining datasets used to train the respective VLMs, to facilitate a fair comparison.

Hierarchical Backbones. When we compare FastVLM (R20) with ConvLLaVA [25] (R18), with the same LLM and similar training data size, our model obtains +8.4% better performance on TextVQA and +12.5% better performance on DocVQA while being 22% faster. The gap widens at higher resolution, where FastVLM (R28 and R29) achieves superior performance on wide range of benchmarks while being 2× faster than ConvLLaVA (R26), with the same LLM decoder.

Dataset Scaling. When scaling the pretraining dataset by incorporating an intermediate pretraining stage for resolution scaling with 15M samples, FastVLM (R21) matches or surpasses MM1 [61] (R38) across a wide range of benchmarks. Remarkably, FastVLM achieves this performance while generating 5× fewer visual tokens. With an input resolution of 1024×1024 and a larger instruction tuning dataset of size 12.5M, FastVLM (R41) outperforms MM1 (R38) and LLaVA-NeXT (R39) across various benchmarks, including text-rich evaluations, like TextVQA and DocVQA, which are sensitive to input resolution and number of visual tokens. The gap widens when further scale up input resolution using AnyRes, more details in Sec. C.1. We provide details of the dataset splits in Sec. D.

Multiple Vision Encoders. Recently, MiniGemini [45] and Cambrian-1 [73] introduced models that rely on multiple vision encoders. In Tab. 6, we compare FastVLM (R40), which uses a single vision encoder with methods that use multiple encoders and trained on similarly scaled visual instruction tuning dataset. In Cambrian-1 [73] (R44), vision encoding contributes 3.2× more than LLM prefilling to the total time-to-first-token of approximately 5 seconds (detailed breakdown is provided in Tab. 9). FastVLM (R40) outperforms Cambrian-1 (R44) when trained on a similar visual instruction tuning dataset, while being 7.9× faster. By scaling the instruction tuning dataset to 12.5M, FastVLM (R41) achieves superior performance over Cambrian-1 (R44) with 2.3× fewer visual tokens, even on text-rich evaluations (see Tab. 10) that are sensitive to the number of visual tokens.

Effect of Decoder. VLM performance also depends on the quality of LLM, as demonstrated in prior studies, like [40]. By switching from Vicuna-7B (R21, R29) to Qwen2 [72, 80] models (R22, R30), we see improvement in performance across all the benchmarks. The improvements are significant on MMVet, LLaVA-in-the-wild and MMMU benchmarks. With Qwen2-0.5B as the LLM decoder, FastVLM (R4) outperforms LLaVA-OneVision [41] (R2) while being 85× faster. This result underscores the quality of our vision encoder, as both models use the same LLM decoder, while FastViTHD is 3.4× smaller compared to SigLIP-SO400M [88].

5. Conclusion

In this work, we introduced FastVLM, which leverages the FastViTHD vision backbone for efficient encoding of high-resolution inputs. FastViTHD has a hybrid architecture, is pretrained on reinforced image-text data, and outputs a substantially reduced number of visual tokens with minimal accuracy sacrifice. FastVLM has competitive performance with prior works across a wide range of VLM benchmarks while improving efficiency in both time-to-first-token and the number of parameters in the vision backbone. Rigorous benchmarking on an M1 MacBook Pro demonstrates that FastVLM achieves a state-of-the-art resolution-latency-accuracy trade-off compared to existing works.

References

[1] Jean-Baptiste Alayrac, Jeff Donahue, Pauline Luc, Antoine Miech, Iain Barr, Yana Hasson, Karel Lenc, Arthur Mensch, Katherine Millican, Malcolm Reynolds, Roman Ring, Eliza Rutherford, Serkan Cabi, Tengda Han, Zhitao Gong, Sina Samangooei, Marianne Monteiro, Jacob L. Menick, Sebastian Borgeaud, Andy Brock, Aida Nematzadeh, Sahand Sharifzadeh, Mikolaj Binkowski, Ricardo Barreira, Oriol Vinyals, Andrew Zisserman, and Karén Simonyan. Flamingo: a visual language model for few-shot learning. *Advances in neural information processing systems*, 2022. 2

[2] Anas Awadalla, Irena Gao, Josh Gardner, Jack Hessel, Yusuf Hanafy, Wanrong Zhu, Kalyani Marathe, Yonatan Bitton, Samir Gadre, Shiori Sagawa, Jenia Jitsev, Simon Kornblith, Pang Wei Koh, Gabriel Ilharco, Mitchell Wortsman, and Ludwig Schmidt. Openflamingo: An open-source framework for training large autoregressive vision-language models. *arXiv preprint arXiv:2308.01390*, 2023. 2

[3] Jinze Bai, Shuai Bai, Yunfei Chu, Zeyu Cui, Kai Dang, Xiaodong Deng, Yang Fan, Wenbin Ge, Yu Han, Fei Huang, Binyuan Hui, Luo Ji, Mei Li, Junyang Lin, Runji Lin, Dayiheng Liu, Gao Liu, Chengqiang Lu, Keming Lu, Jianxin Ma, Rui Men, Xingzhang Ren, Xuancheng Ren, Chuanqi Tan, Sinan Tan, Jianhong Tu, Peng Wang, Shijie Wang, Wei Wang, Shengguang Wu, Benfeng Xu, Jin Xu, An Yang, Hao Yang, Jian Yang, Shusheng Yang, Yang Yao, Bowen Yu, Hongyi Yuan, Zheng Yuan, Jianwei Zhang, Xingxuan Zhang, Yichang Zhang, Zhenru Zhang, Chang Zhou, Jingren Zhou, Xiaohuan Zhou, and Tianhang Zhu. Qwen technical report. *arXiv preprint arXiv:2309.16609*, 2023. 7, 2

[4] Jinze Bai, Shuai Bai, Shusheng Yang, Shijie Wang, Sinan Tan, Peng Wang, Junyang Lin, Chang Zhou, and Jingren Zhou. Qwen-vl: A versatile vision-language model for understanding, localization, text reading, and beyond. *arXiv preprint arXiv:2308.12966*, 202k. 2, 7

[5] Rohan Bavishi, Erich Elsen, Curtis Hawthorne, Maxwell Nye, Augustus Odena, Arushi Somani, and Sağnak Taşırlar. Introducing our multimodal models, 2023. 2

[6] Lucas Beyer, Andreas Steiner, André Susano Pinto, Alexander Kolesnikov, Xiao Wang, Daniel Salz, Maxim Neumann, Ibrahim Alabdulmohsin, Michael Tschannen, Emanuele Bugliarello, Thomas Unterthiner, Daniel Keysers, Skanda Koppula, Fangyu Liu, Adam Grycner, Alexey Gritsenko, Neil Houlsby, Manoj Kumar, Keran Rong, Julian Eisenschlos, Rishabh Kabra, Matthias Bauer, Matko Bošnjak, Xi Chen, Matthias Minderer, Paul Voigtlaender, Ioana Bica, Ivana Balazevic, Joan Puigcerver, Pinelopi Papalampidi, Olivier Henaff, Xi Xiong, Radu Soricut, Jeremiah Harmsen, and Xiaohua Zhai. Paligemma: A versatile 3b vlm for transfer, 2024. 2

[7] Mu Cai, Jianwei Yang, Jianfeng Gao, and Yong Jae Lee. Matryoshka multimodal models. *arXiv preprint arXiv:2405.17430*, 2024. 3, 6

[8] Jie "Cao and Jing" Xiao. "an augmented benchmark dataset for geometric question answering through dual parallel text encoding". In *"Proceedings of the 29th International Conference on Computational Linguistics"*, "2022". 3

[9] Junbum Cha, Wooyoung Kang, Jonghwan Mun, and Byungseok Roh. Honeybee: Locality-enhanced projector for multimodal llm. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024. 3

[10] Soravit Changpinyo, Piyush Sharma, Nan Ding, and Radu Soricut. Conceptual 12m: Pushing web-scale image-text pre-training to recognize long-tail visual concepts. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 3558–3568, 2021. 2

[11] Jieneng Chen, Qihang Yu, Xiaohui Shen, Alan Yuille, and Liang-Chieh Chen. Vitamin: Designing scalable vision models in the vision-language era. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2024. 4, 7, 1, 2

[12] Lin Chen, Jisong Li, Xiaoyi Dong, Pan Zhang, Conghui He, Jiaqi Wang, Feng Zhao, and Dahua Lin. Sharegpt4v: Improving large multi-modal models with better captions. *arXiv preprint arXiv:2311.12793*, 2023. 7, 2

[13] Liang Chen, Haozhe Zhao, Tianyu Liu, Shuai Bai, Junyang Lin, Chang Zhou, and Baobao Chang. An image is worth 1/2 tokens after layer 2: Plug-and-play inference acceleration for large vision-language models, 2024. 6

[14] Zhe Chen, Jiannan Wu, Wenhai Wang, Weijie Su, Guo Chen, Sen Xing, Muyan Zhong, Qinglong Zhang, Xizhou Zhu, Lewei Lu, Bin Li, Ping Luo, Tong Lu, Yu Qiao, and Jifeng Dai. Internvl: Scaling up vision foundation models and aligning for generic visual-linguistic tasks. *arXiv preprint arXiv:2312.14238*, 2023. 2, 3, 4

[15] Zhe Chen, Weiyun Wang, Hao Tian, Shenglong Ye, Zhangwei Gao, Erfei Cui, Wenwen Tong, Kongzhi Hu, Jiapeng Luo, Zheng Ma, et al. How far are we to gpt-4v? closing the gap to commercial multimodal models with open-source suites. *arXiv preprint arXiv:2404.16821*, 2024. 2

[16] Chenglin Yang et al. MOAT: Alternating mobile convolution and attention brings strong vision models. In *ICLR*, 2023. 4

[17] Xiangxiang Chu, Limeng Qiao, Xinyang Lin, Shuang Xu, Yang Yang, Yiming Hu, Fei Wei, Xinyu Zhang, Bo Zhang, Xiaolin Wei, et al. Mobilevlm: A fast, reproducible and strong vision language assistant for mobile devices. *arXiv preprint arXiv:2312.16886*, 2023. 3, 7, 2

[18] Xiangxiang Chu, Limeng Qiao, Xinyu Zhang, Shuang Xu, Fei Wei, Yang Yang, Xiaofei Sun, Yiming Hu, Xinyang Lin, Bo Zhang, et al. Mobilevlm v2: Faster and stronger baseline for vision language model. *arXiv preprint arXiv:2402.03766*, 2024. 7, 2

[19] Wenliang Dai, Junnan Li, Dongxu Li, Anthony Meng Huat Tiong, Junqi Zhao, Weisheng Wang, Boyang Li, Pascale Fung, and Steven Hoi. Instructblip: Towards general-purpose vision-language models with instruction tuning, 2023. 2, 3, 7

[20] DeepSeek-AI. Deepseek llm: Scaling open-source language models with longtermism. *arXiv preprint arXiv:2401.02954*, 2024. 7, 2

[21] Haiwen Diao, Yufeng Cui, Xiaotong Li, Yueze Wang, Huchuan Lu, and Xinlong Wang. Unveiling encoder-free vision-language models. *arXiv preprint arXiv:2406.11832*, 2024. 2

[22] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, et al. An image is worth 16x16 words: Transformers for image recognition at scale. *arXiv preprint arXiv:2010.11929*, 2020. 2, 4

[23] Alex Fang, Albin Madappally Jose, Amit Jain, Ludwig Schmidt, Alexander Toshev, and Vaishaal Shankar. Data filtering networks. *arXiv preprint arXiv:2309.17425*, 2023. 3

[24] Samir Yitzhak Gadre, Gabriel Ilharco, Alex Fang, Jonathan Hayase, Georgios Smyrnis, Thao Nguyen, Ryan Marten, Mitchell Wortsman, Dhruba Ghosh, Jieyu Zhang, et al. Datacomp: In search of the next generation of multimodal datasets. *arXiv preprint arXiv:2304.14108*, 2023. 4

[25] Chunjiang Ge, Sijie Cheng, Ziming Wang, Jiale Yuan, Yuan Gao, Jun Song, Shiji Song, Gao Huang, and Bo Zheng. Convllava: Hierarchical backbones as visual encoder for large multimodal models, 2024. 2, 3, 7, 8

[26] Yash Goyal, Tejas Khot, Douglas Summers-Stay, Dhruv Batra, and Devi Parikh. Making the v in vqa matter: Elevating the role of image understanding in visual question answering. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 6904–6913, 2017. 8

[27] Awni Hannun, Jagrit Digani, Angelos Katharopoulos, and Ronan Collobert. MLX: Efficient and flexible machine learning on apple silicon, 2023. 7, 8, 3

[28] Wenbo Hu, Zi-Yi Dou, Liunian Harold Li, Amita Kamath, Nanyun Peng, and Kai-Wei Chang. Matryoshka query transformer for large vision-language models, 2024. 3, 6

[29] Wenxuan Huang, Zijie Zhai, Yunhang Shen, Shaoshen Cao, Fei Zhao, Xiangfeng Xu, Zheyu Ye, and Shaohui Lin. Dynamic-llava: Efficient multimodal large language models via dynamic vision-language context sparsification, 2024. 6

[30] Drew A Hudson and Christopher D Manning. Gqa: A new dataset for real-world visual reasoning and compositional question answering. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 6700–6709, 2019. 6, 5

[31] Kushal Kafle, Scott Cohen, Brian Price, and Christopher Kanan. Dvqa: Understanding data visualizations via question answering. In *CVPR*, 2018. 3

[32] Siddharth Karamcheti, Suraj Nair, Ashwin Balakrishna, Percy Liang, Thomas Kollar, and Dorsa Sadigh. Prismatic vlms: Investigating the design space of visually-conditioned language models. In *International Conference on Machine Learning (ICML)*, 2024. 3

[33] Aniruddha Kembhavi, Mike Salvato, Eric Kolve, Minjoon Seo, Hannaneh Hajishirzi, and Ali Farhadi. A diagram is worth a dozen images, 2016. 3

[34] Geewook Kim, Teakgyu Hong, Moonbin Yim, JeongYeon Nam, Jinyoung Park, Jinyeong Yim, Wonseok Hwang, Sangdoo Yun, Dongyoon Han, and Seunghyun Park. Ocr-free document understanding transformer. In *European Conference on Computer Vision (ECCV)*, 2022. 3

[35] Alexander Kirillov, Eric Mintun, Nikhila Ravi, Hanzi Mao, Chloe Rolland, Laura Gustafson, Tete Xiao, Spencer Whitehead, Alexander C. Berg, Wan-Yen Lo, Piotr Dollár, and Ross Girshick. Segment anything. *arXiv preprint arXiv:2304.02643*, 2023. 3

[36] Ranjay Krishna, Yuke Zhu, Oliver Groth, Justin Johnson, Kenji Hata, Joshua Kravitz, Stephanie Chen, Yannis Kalantidis, Li-Jia Li, David A. Shamma, Michael S. Bernstein, and Li Fei-Fei. Visual genome: Connecting language and vision using crowdsourced dense image annotations. *International Journal of Computer Vision*, 2017. 3

[37] Hugo Laurençon, Andrés Marafioti, Victor Sanh, and Léo Tronchon. Building and better understanding vision-language models: insights and future directions., 2024. 4

[38] Bohao Li, Rui Wang, Guangzhi Wang, Yuying Ge, Yixiao Ge, and Ying Shan. Seed-bench: Benchmarking multimodal llms with generative comprehension. *arXiv preprint arXiv:2307.16125*, 2023. 8

[39] Bo Li, Hao Zhang, Kaichen Zhang, Dong Guo, Yuanhan Zhang, Renrui Zhang, Feng Li, Ziwei Liu, and Chunyuan Li. Llava-next: What else influences visual instruction tuning beyond data?, 2024. 6, 1, 2, 3

[40] Bo Li, Kaichen Zhang, Hao Zhang, Dong Guo, Renrui Zhang, Feng Li, Yuanhan Zhang, Ziwei Liu, and Chunyuan Li. Llava-next: Stronger llms supercharge multimodal capabilities in the wild, 2024. 8

[41] Bo Li, Yuanhan Zhang, Dong Guo, Renrui Zhang, Feng Li, Hao Zhang, Kaichen Zhang, Yanwei Li, Ziwei Liu, and Chunyuan Li. Llava-onevision: Easy visual task transfer. *arXiv preprint arXiv:2408.03326*, 2024. 2, 7, 8, 3, 4

[42] Junyan Li, Delin Chen, Tianle Cai, Peihao Chen, Yining Hong, Zhenfang Chen, Yikang Shen, and Chuang Gan. Flexattention for efficient high-resolution vision-language models. In *European Conference on Computer Vision*, pages 286–302. Springer, 2025. 7

[43] Kevin Y. Li, Sachin Goyal, Joao D. Semedo, and J. Zico Kolter. Inference optimal vlms need only one visual token but larger models, 2024. 4

[44] Yifan Li, Yifan Du, Kun Zhou, Jinpeng Wang, Wayne Xin Zhao, and Ji-Rong Wen. Evaluating object hallucination in large vision-language models. *arXiv preprint arXiv:2305.10355*, 2023. 6

[45] Yanwei Li, Yuechen Zhang, Chengyao Wang, Zhisheng Zhong, Yixin Chen, Ruihang Chu, Shaoteng Liu, and Jiaya Jia. Mini-gemini: Mining the potential of multi-modality vision language models. *arXiv preprint arXiv:2403.18814*, 2023. 8

[46] Ji Lin, Hongxu Yin, Wei Ping, Yao Lu, Pavlo Molchanov, Andrew Tao, Huizi Mao, Jan Kautz, Mohammad Shoeybi, and Song Han. Vila: On pre-training for visual language models, 2023. 2, 7

[47] Tsung-Yi Lin, Michael Maire, Serge Belongie, James Hays, Pietro Perona, Deva Ramanan, Piotr Dollár, and C Lawrence Zitnick. Microsoft coco: Common objects in context. In *European Conference on Computer Vision*, pages 740–755, 2014. 3

[48] Ziyi Lin, Chris Liu, Renrui Zhang, Peng Gao, Longtian Qiu, Han Xiao, Han Qiu, Chen Lin, Wenqi Shao, Keqin Chen, et al. Sphinx: The joint mixing of weights, tasks, and visual embeddings for multi-modal large language models. *arXiv preprint arXiv:2311.07575*, 2023. 2, 6

[49] Haotian Liu, Chunyuan Li, Yuheng Li, and Yong Jae Lee. Improved baselines with visual instruction tuning, 2023. 1, 2, 3, 5, 6, 7

[50] Haotian Liu, Chunyuan Li, Qingyang Wu, and Yong Jae Lee. Visual instruction tuning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. 1, 6, 8

[51] Haotian Liu, Chunyuan Li, Yuheng Li, Bo Li, Yuanhan Zhang, Sheng Shen, and Yong Jae Lee. Llava-next: Improved reasoning, ocr, and world knowledge, 2024. 2

[52] Yuliang Liu, Zhang Li, Mingxin Huang, Biao Yang, Wenwen Yu, Chunyuan Li, Xucheng Yin, Cheng lin Liu, Lianwen Jin, and Xiang Bai. Ocrbench: On the hidden mystery of ocr in large multimodal models, 2024. 4

[53] Zhuang Liu, Hanzi Mao, Chao-Yuan Wu, Christoph Feichtenhofer, Trevor Darrell, and Saining Xie. A convnet for the 2020s. *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022. 3

[54] Haoyu Lu, Wen Liu, Bo Zhang, Bingxuan Wang, Kai Dong, Bo Liu, Jingxiang Sun, Tongzheng Ren, Zhuoshu Li, Hao Yang, Yaofeng Sun, Chengqi Deng, Hanwei Xu, Zhenda Xie, and Chong Ruan. Deepseek-vl: Towards real-world vision-language understanding, 2024. 7, 2

[55] Pan Lu, Swaroop Mishra, Tanglin Xia, Liang Qiu, Kai-Wei Chang, Song-Chun Zhu, Oyvind Tafjord, Peter Clark, and Ashwin Kalyan. Learn to explain: Multimodal reasoning via thought chains for science question answering. *Advances in Neural Information Processing Systems*, 2022. 6, 3

[56] Pan Lu, Hritik Bansal, Tony Xia, Jiacheng Liu, Chunyuan Li, Hannaneh Hajishirzi, Hao Cheng, Kai-Wei Chang, Michel Galley, and Jianfeng Gao. Mathvista: Evaluating mathematical reasoning of foundation models in visual contexts. In *International Conference on Learning Representations (ICLR)*, 2024. 2

[57] Gen Luo, Yiyi Zhou, Yuxin Zhang, Xiawu Zheng, Xiaoshuai Sun, and Rongrong Ji. Feast your eyes: Mixture-of-resolution adaptation for multimodal large language models. *arXiv preprint arXiv:2403.03003*, 2024. 2

[58] Ahmed Masry, Do Long, Jia Qing Tan, Shafiq Joty, and Enamul Hoque. ChartQA: A benchmark for question answering about charts with visual and logical reasoning. In *"Findings of the Association for Computational Linguistics: ACL 2022"*, "2022". 3, 4, 5

[59] Minesh Mathew, Viraj Bagal, Rubèn Pérez Tito, Dimosthenis Karatzas, Ernest Valveny, and C. V Jawahar. Infographicvqa, 2021. 4

[60] Minesh Mathew, Dimosthenis Karatzas, and CV Jawahar. Docvqa: A dataset for vqa on document images. In *Proceedings of the IEEE/CVF winter conference on applications of computer vision*, pages 2200–2209, 2021. 8, 3, 5

[61] Brandon McKinzie, Zhe Gan, Jean-Philippe Fauconnier, Sam Dodge, Bowen Zhang, Philipp Dufter, Dhruti Shah, Xianzhi Du, Futang Peng, Floris Weers, Anton Belyi, Haotian Zhang, Karanjeet Singh, Doug Kang, Ankur Jain, Hongyu Hè, Max Schwarzer, Tom Gunter, Xiang Kong, Aonan Zhang, Jianyu Wang, Chong Wang, Nan Du, Tao Lei, Sam Wiseman, Guoli Yin, Mark Lee, Zirui Wang, Ruoming Pang, Peter Grasch, Alexander Toshev, and Yinfei Yang. Mm1: Methods, analysis & insights from multimodal llm pre-training, 2024. 2, 6, 7, 8, 3

[62] Anand Mishra, Shashank Shekhar, Ajeet Kumar Singh, and Anirban Chakraborty. Ocr-vqa: Visual question answering by reading text in images. In *ICDAR*, 2019. 3

[63] OpenAI. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774*, 2023. 2

[64] Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, et al. Learning transferable visual models from natural language supervision. In *International conference on machine learning*, pages 8748–8763. PMLR, 2021. 1, 2

[65] Yuzhang Shang, Mu Cai, Bingxin Xu, Yong Jae Lee, and Yan Yan. Llava-prumerge: Adaptive token reduction for efficient large multimodal models. *arXiv preprint arXiv:2403.15388*, 2024. 3, 6

[66] Piyush Sharma, Nan Ding, Sebastian Goodman, and Radu Soricut. Conceptual captions: A cleaned, hypernymed, image alt-text dataset for automatic image captioning. In *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 2556–2565, 2018. 2

[67] Baifeng Shi, Ziyang Wu, Maolin Mao, Xin Wang, and Trevor Darrell. When do we not need larger vision models? In *European Conference on Computer Vision (ECCV)*, 2024. 2

[68] Min Shi, Fuxiao Liu, Shihao Wang, Shijia Liao, Subhashree Radhakrishnan, De-An Huang, Hongxu Yin, Karan Sapra, Yaser Yacoob, Humphrey Shi, Bryan Catanzaro, Andrew Tao, Jan Kautz, Zhiding Yu, and Guilin Liu. Eagle: Exploring the design space for multimodal llms with mixture of encoders. *arXiv:2408.15998*, 2024. 3

[69] Amanpreet Singh, Vivek Natarajan, Meet Shah, Yu Jiang, Xinlei Chen, Dhruv Batra, Devi Parikh, and Marcus Rohrbach. Towards vqa models that can read. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 8317–8326, 2019. 6, 3

[70] Quan Sun, Yuxin Fang, Ledell Wu, Xinlong Wang, and Yue Cao. Eva-clip: Improved training techniques for clip at scale. *arXiv preprint arXiv:2303.15389*, 2023. 3

[71] Chameleon Team. Chameleon: Mixed-modal early-fusion foundation models, 2024. 2

[72] Qwen Team. Qwen2.5: A party of foundation models, 2024. 2, 8

[73] Shengbang Tong, Ellis Brown, Penghao Wu, Sanghyun Woo, Manoj Middepogu, Sai Charitha Akula, Jihan Yang, Shusheng Yang, Adithya Iyer, Xichen Pan, Austin Wang, Rob Fergus, Yann LeCun, and Saining Xie. Cambrian-1: A fully open, vision-centric exploration of multimodal llms, 2024. 2, 3, 7, 8

[74] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, Aurelien Rodriguez, Armand Joulin, Edouard Grave, and Guillaume Lample. Llama: Open and efficient foundation language models, 2023. 2

[75] Maria Tsimpoukelli, Jacob Menick, Serkan Cabi, SM Eslami, Oriol Vinyals, and Felix Hill. Multimodal few-shot

learning with frozen language models. *Conference on Neural Information Processing Systems (NeurIPS)*, 2021. 2

[76] Pavan Kumar Anasosalu Vasu, James Gabriel, Jeff Zhu, Oncel Tuzel, and Anurag Ranjan. Mobileone: An improved one millisecond mobile backbone. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2023. 1

[77] Pavan Kumar Anasosalu Vasu, James Gabriel, Jeff Zhu, Oncel Tuzel, and Anurag Ranjan. Fastvit: A fast hybrid vision transformer using structural reparameterization. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2023. 2, 3, 4, 1

[78] Pavan Kumar Anasosalu Vasu, Hadi Pouransari, Fartash Faghri, Raviteja Vemulapalli, and Oncel Tuzel. Mobileclip: Fast image-text models through multi-modal reinforced training. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024. 2, 3, 4, 1

[79] Le Xue, Manli Shu, Anas Awadalla, Jun Wang, An Yan, Senthil Purushwalkam, Honglu Zhou, Viraj Prabhu, Yutong Dai, Michael S. Ryoo, Shrikant Kendre, Jieyu Zhang, Can Qin, Shu Zhang, Chia-Chih Chen, Ning Yu, Juntao Tan, Tulika Manoj Awalgaonkar, Shelby Heinecke, Huan Wang, Yejin Choi, Ludwig Schmidt, Zeyuan Chen, Silvio Savarese, Juan Carlos Niebles, Caiming Xiong, and Ran Xu. xgen-mm (BLIP-3): A family of open large multimodal models. *CoRR*, abs/2408.08872, 2024. 2

[80] An Yang, Baosong Yang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Zhou, Chengpeng Li, Chengyuan Li, Dayiheng Liu, Fei Huang, Guanting Dong, Haoran Wei, Huan Lin, Jialong Tang, Jialin Wang, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Ma, Jin Xu, Jingren Zhou, Jinze Bai, Jinzheng He, Junyang Lin, Kai Dang, Keming Lu, Keqin Chen, Kexin Yang, Mei Li, Mingfeng Xue, Na Ni, Pei Zhang, Peng Wang, Ru Peng, Rui Men, Ruize Gao, Runji Lin, Shijie Wang, Shuai Bai, Sinan Tan, Tianhang Zhu, Tianhao Li, Tianyu Liu, Wenbin Ge, Xiaodong Deng, Xiaohuan Zhou, Xingzhang Ren, Xinyu Zhang, Xipin Wei, Xuancheng Ren, Yang Fan, Yang Yao, Yichang Zhang, Yu Wan, Yunfei Chu, Yuqiong Liu, Zeyu Cui, Zhenru Zhang, and Zhihao Fan. Qwen2 technical report. *arXiv preprint arXiv:2407.10671*, 2024. 1, 2, 5, 6, 7, 8, 3

[81] Senqiao Yang, Yukang Chen, Zhuotao Tian, Chengyao Wang, Jingyao Li, Bei Yu, and Jiaya Jia. Visionzip: Longer is better but not necessary in vision language models, 2024. 6

[82] Jiabo Ye, Haiyang Xu, Haowei Liu, Anwen Hu, Ming Yan, Qi Qian, Ji Zhang, Fei Huang, and Jingren Zhou. mplug-owl3: Towards long image-sequence understanding in multi-modal large language models, 2024. 2

[83] Qinghao Ye, Haiyang Xu, Guohai Xu, Jiabo Ye, Ming Yan, Yiyang Zhou, Junyang Wang, Anwen Hu, Pengcheng Shi, Yaya Shi, Chaoya Jiang, Chenliang Li, Yuanhong Xu, Hehong Chen, Junfeng Tian, Qian Qi, Ji Zhang, and Fei Huang. mplug-owl: Modularization empowers large language models with multimodality, 2023.

[84] Qinghao Ye, Haiyang Xu, Jiabo Ye, Ming Yan, Anwen Hu, Haowei Liu, Qi Qian, Ji Zhang, Fei Huang, and Jingren Zhou. mplug-owl2: Revolutionizing multi-modal large language model with modality collaboration, 2023. 2

[85] Weihao Yu, Zhengyuan Yang, Linjie Li, Jianfeng Wang, Kevin Lin, Zicheng Liu, Xinchao Wang, and Lijuan Wang. Mm-vet: Evaluating large multimodal models for integrated capabilities. *arXiv preprint arXiv:2308.02490*, 2023. 8

[86] Weihao Yu, Chenyang Si, Pan Zhou, Mi Luo, Yichen Zhou, Jiashi Feng, Shuicheng Yan, and Xinchao Wang. Metaformer baselines for vision. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2024. 4

[87] Xiang Yue, Yuansheng Ni, Kai Zhang, Tianyu Zheng, Ruoqi Liu, Ge Zhang, Samuel Stevens, Dongfu Jiang, Weiming Ren, Yuxuan Sun, Cong Wei, Botao Yu, Ruibin Yuan, Renliang Sun, Ming Yin, Boyuan Zheng, Zhenzhu Yang, Yibo Liu, Wenhao Huang, Huan Sun, Yu Su, and Wenhu Chen. Mmmu: A massive multi-discipline multimodal understanding and reasoning benchmark for expert agi. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 9556–9567, 2024. 8, 4

[88] Xiaohua Zhai, Basil Mustafa, Alexander Kolesnikov, and Lucas Beyer. Sigmoid loss for language image pre-training. *International Conference on Computer Vision (ICCV)*, 2023. 2, 3, 8

[89] Kaichen Zhang, Bo Li, Peiyuan Zhang, Fanyi Pu, Joshua Adrian Cahyono, Kairui Hu, Shuai Liu, Yuanhan Zhang, Jingkang Yang, Chunyuan Li, and Ziwei Liu. Lmms-eval: Reality check on the evaluation of large multimodal models, 2024. 8

[90] Yuan Zhang, Chun-Kai Fan, Junpeng Ma, Wenzhao Zheng, Tao Huang, Kuan Cheng, Denis Gudovskiy, Tomoyuki Okuno, Yohei Nakata, Kurt Keutzer, et al. Sparsevlm: Visual token sparsification for efficient vision-language model inference. *arXiv preprint arXiv:2410.04417*, 2024. 6

[91] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric. P Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. Judging llm-as-a-judge with mt-bench and chatbot arena, 2023. 1, 2, 3, 6, 7

[92] Deyao Zhu, Jun Chen, Xiaoqian Shen, Xiang Li, and Mohamed Elhoseiny. Minigpt-4: Enhancing vision-language understanding with advanced large language models. *arXiv preprint arXiv:2304.10592*, 2023. 2